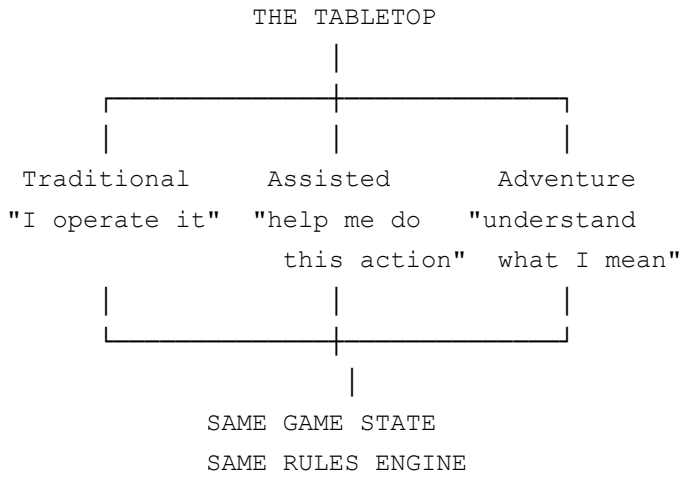
Intelligent Tabletop — UX/UI Design Blueprint

Status: Design specification only. No application code is modified by this document.
Audience: The implementation agent that will eventually build against this spec.
Grounding: This blueprint evolves the current prototype's proven direction (a dark-wood-and-parchment fantasy tabletop, Cinzel/serif typography, brass-gold accents, a three-region layout) rather than replacing it. Where the current build already gets something right, that is called out explicitly and kept. Where it has a real gap — scalability past a handful of abilities, responsive behavior, targeting legibility at scale — this document designs the fix.

---

0. Product Framing

The tabletop is one surface with three input methods, not three products glued together:



Every design decision below is checked against one question: **does this make the mode switcher feel like a lens, or like a different app?** Anything that makes Assisted/Adventure look like a chat window bolted onto a game fails that test.

---

1. Overall Screen Architecture

Visual hierarchy, most → least important:

1. **The board.** Always the largest element on screen, always visually centered, never obscured by a panel that can't be dismissed.
2. **Whose turn it is / who I'm controlling.** This must be answerable in under one second of looking at the screen — active-actor state is the second most important thing after the board itself.
3. **What I can currently do.** The action bar / command bar, positioned so it's discoverable without hunting.
4. **Party status.** HP and readiness for both party members, always visible, never behind a click.
5. **Enemy status.** Visible, but secondary to party status — the player manages their own resources more often than they audit the enemy's.
6. **Turn order.** Useful, but glanceable, not a primary focus.
7. **Session log.** Present and readable, but the least visually dominant surface. It's a record, not a control.

What stays visible without opening a secondary panel, at all times:

- The full board
- Current turn indicator
- Active character's HP/AC/available actions
- The other party member's HP at a glance
- Enemy HP for anything currently visible on the board
- The mode switcher (Traditional / Assisted / Adventure)

What can legitimately move to a secondary surface (see §12 for responsive rules):

- Full initiative order beyond "current + next"
- Full session log history beyond the last few events
- Detailed enemy stat blocks (weapon, exact AC breakdown) — a hover/tap reveal, not always-on

This matches the current build's instinct (three-region layout: party+actions on the left, board center, initiative+log on the right) — that skeleton is correct and should be kept. The refinement is making each region gracefully collapse under real width constraints instead of assuming desktop width always exists.

2. Board Experience

The board is the tabletop's face. It needs a small, consistent, learnable set of visual states rather than a large decorative vocabulary.

Token visual states:

State	Treatment
Default	Resting appearance, no border emphasis, side-colored fill (party vs. enemy hue)
Selected	Distinct outline (currently gold) — the character *whose panel is currently open*, not necessarily whose turn it is
Active turn	A second, independent signal — the current build conflates "selected" and "current actor" a bit at the panel level; the board token needs its own always-on "it's your move" affordance (e.g., a subtle pulse or a turn-order-colored ring) so that when a player deselects to inspect an enemy, they don't lose track of whose turn it still is
Dead / defeated	Token removed from the board (current behavior is correct) or, if a "defeated but visible" concept is ever needed later, a desaturated/prone treatment — not needed now

Tile visual states, layered independently (a tile can be more than one at once):

State	Treatment
Terrain (open floor)	Base texture, no emphasis
Wall	Solid, clearly impassable material
Pillar / cover object	Visually distinct from wall — reads as "an object in the room," not "the edge of the room"
Reachable (movement pending)	Soft inset highlight (current green-tinted inset is a good, non-noisy pattern — keep it)
Attack target (valid)	A different hue from movement (current build uses a red-tinted highlight on the token itself, which is correct — attack should never look like the friendly "move here" green)
	A third, ability-appropriate hue — currently a green ring is reused

Ability target (valid)	for ability targeting, which risks being confused with movement's green. **Recommendation:** give beneficial-ability targeting (heal) a distinct cool tone (e.g., soft blue) and harmful-ability targeting (damage spells) the same warning hue attacks use, since both are "aim at something" gestures the player should read the same way attacks do
Invalid target (hovered/attempted)	Not a board-tile treatment — this is a feedback event, not a resting state (see §6)
Line of sight indicator	Not shown as a persistent overlay (would be visual noise); revealed contextually — see §6

Principle: movement highlighting, attack highlighting, and ability highlighting must never share a color, because the player needs to instantly tell "where can I walk" from "what can I hurt" from "who can I help" without reading text.

---

3. Character / Party Presentation

Each party member's panel must answer two questions at a glance: **who am I controlling, and can they still act?**

Per-character panel contents, in priority order:

1. Name and class (identity)
2. HP / max HP, shown as both a number and a bar (numbers for precision, bar for at-a-glance read)
3. Active-turn indicator (a border/glow treatment distinct from "selected" — see §2's point about not conflating these)
4. Selection state (does clicking this panel show its controls?)
5. Action-availability state: a clear, non-ambiguous "ready" vs. "action used" signal — this is one of the most important pieces of state in the whole UI, because it directly gates what the action bar will let the player do
6. AC and movement-remaining as smaller, secondary stats — useful, not load-bearing for the primary glance
7. Portrait — a real product would want character art here; the current build's icon-in-a-circle approach is an acceptable placeholder and should stay schematic (not attempt fantasy portrait art) until real art assets exist, so the interface doesn't look

"half-finished" by having one polished portrait and three programmer-art ones

Answering "who am I controlling": the active-turn state and the selected state need to be visually distinguishable from each other but complementary — when they coincide (the normal case: you select the character whose turn it is), the panel should read unambiguously as "this one, right now." When a player deselects the active character to go inspect an enemy panel, the active-turn treatment must persist on the original panel so they don't lose the thread.

---

4. Turn Order

Keep it compact — a horizontal or vertical strip of small tokens/chips, not a table. Requirements:

- **Current actor** clearly marked (an arrow/chevron pointing at it, as the current build already does, is a good pattern)
- **Next actor** visually next in reading order — no reordering/scrolling required to see who's up next
- **Player vs. enemy** distinguished by the same side-coloring used on the board, so the strip and the board reinforce each other rather than introducing a second color language
- **Defeated combatants** shown struck-through/dimmed in place rather than removed from the strip — removing them would make the turn order visually jump, which is disorienting during a fast combat

Turn transitions: when control passes from the player to an enemy, the turn-order strip's active marker should move *before* the enemy's actions resolve, so the player understands "it's their turn now" as a distinct beat, not something they infer after the fact from the log. When control returns to the player, the marker's arrival at their token is the cue — reinforced by the action bar becoming interactive again (see §14 for the accompanying animation notes).

---

5. Action Bar

Current concept (Move / Attack / Healing Touch / Fire Bolt / End Turn) is a 2-ability case. The design needs to hold up at 8-10 abilities without becoming a wall of buttons.

Structural recommendation — two tiers:

- **Tier 1 (always visible, fixed):** Move, Attack, End Turn. These are universal, every combatant has them, and they should never scroll or collapse.
- **Tier 2 (data-driven, scrollable/wrapping row):** abilities, exactly as the current build already does — one button per entry in the character's ability list, generated generically, not hand-coded per ability. This is already the right architecture underneath; the *visual* container just needs a defined behavior for overflow: wrap to a second row up to some row-count budget (2 rows), then switch to a horizontally scrollable strip past that, rather than letting the panel grow indefinitely and pushing End Turn off-screen. This is the direct fix for the "many abilities eventually" scalability concern.

Button states:

- **Default/available:** normal contrast, clearly clickable
- **Selected/pending** (i.e., "I am now choosing a target for this"): strong, unambiguous highlight — the current gold-fill toggle state is a good pattern, keep it
- **Disabled — action already used:** visually muted, but the button should remain visible (not hidden) with a tooltip explaining why, rather than disappearing — a player should never wonder "where did Attack go," they should see it, dimmed, with a reason on hover
- **Disabled — not your turn:** the entire action bar for a non-active character simply doesn't render (current behavior), which is correct — there's no ambiguity to design for there

Tooltips: every ability button should support a hover/long-press tooltip showing at minimum: range, and one-line effect summary (e.g., "Fire Bolt — Range 4, d8+1 damage, requires line of sight"). This is pulled directly from the ability's own data — the UI reads it, it doesn't hardcode it — so new abilities automatically get correct tooltips.

Input parity: every action-bar interaction (select action → click target) must have a mouse, touch, and command-bar equivalent producing the *same proposal shape* downstream. Keyboard shortcuts (e.g., number keys 1-5 for ability slots) are a reasonable desktop enhancement but not required for v1 — touch and mouse parity is the hard requirement, keyboard is a nice-to-have layered on top later.

---

6. Targeting Experience

This is the highest-leverage section for making the tabletop feel like it "understands the game," because it's where Traditional Mode either does or doesn't visibly share intelligence with Assisted/Adventure.

Sequence: player clicks "Attack," then "Fire Bolt":

1. **On selecting an action/ability**, the board should *immediately* recompute and show every legal target for that specific action — not a generic "here are all enemies" list, but the literal output of the same validation function that will run when they click. If Fire Bolt has range 4 and requires LOS, only targets satisfying both light up; a goblin at range 6 does not, even though it's an enemy.
2. **Targets are grouped into exactly two visual buckets on the board**, no more: *valid* (can click, will resolve) and *everything else* (default appearance, unhighlighted). Do not pre-render six different "why invalid" colors on the board itself — that's the visual-noise failure mode the brief warns against. The board answers "what can I click," binary and fast.
3. **The "why not" answer is a feedback event, not a resting state.** If the player clicks something that isn't a valid target, *that's* when the specific reason appears — as a short-lived, dismissible banner using the actual validation reason (already how the current build behaves, and it's the right pattern: "Aldric is not a valid target for Fire Bolt," not a generic "invalid"). This keeps the board itself clean while still answering "why can't I target it?" the moment the player actually tries.
4. **Hover/long-press preview (recommended addition):** before committing to a click, a light hover state on a valid target can preview the action's name and target ("Fire Bolt → Goblin 1") in a small tooltip near the cursor/finger — this answers "what will happen if I click?" without requiring a click to find out, and costs nothing architecturally since it's just displaying the same validation result the click would have used.

Distinguishing failure reasons, all surfaced through the same feedback-banner mechanism (never invented text, always the rules engine's own reason):

- Out of range → distance shown in the reason ("5 > 4")
- Blocked LOS → named explicitly, distinct from "out of range" so the player learns the tactical difference between "too far" and "something's in the way"
- Wrong side (targeting an ally with an enemy-only ability, or vice versa) → named as a targeting-rule violation, not lumped in with range/LOS
- Defeated target → can't occur through the board (dead tokens are removed), but is relevant for command-bar input referencing a dead combatant by name — same

banner mechanism applies

Movement targeting follows the same two-bucket model: reachable tiles highlighted, everything else default. Clicking a non-reachable tile that's nonetheless a "real" destination (blocked, occupied) should produce the same kind of reason-banner as an invalid attack — this is a known current gap (occupied-tile feedback is currently only reachable by clicking a tile's uncovered margin, not the token sitting on it) and should be explicitly designed for: **a token occupying a tile should not fully consume that tile's click target when the player is in Move mode.** Concretely: while an action other than "select this character" is pending (Move/Attack/Ability), clicks on any token should be interpreted through the pending action's targeting rules first, and only fall back to reselection if the pending action doesn't apply to that token at all.

---

7. Command Bar

The command bar is a control surface for the tabletop, not a chat window. Design cues that reinforce this:

- **No avatar, no "typing" indicator, no message bubbles.** Input and system response render as tabletop UI (cards, banners), not chat turns.
- **Placeholder text is content-driven** (already correct in the current build — it names whatever enemy actually exists in the encounter, e.g., "attack the orc" in Training Yard vs. "attack the goblin" in the Crypt) — this small detail matters because a static placeholder mentioning a monster that isn't in the current encounter breaks the illusion that the system is actually looking at the board.
- **Submit interaction:** Enter key and an explicit button, both present — touch users need the button, desktop users expect Enter.
- **History:** an up-arrow-recallable input history (last N commands) is a low-cost, high-value addition not present in the current build — useful for players iterating on phrasing.
- **Suggestions:** a persistent, small "try:" hint row beneath the input (current build has this) rather than an autocomplete dropdown — autocomplete implies the system is guessing as you type, which oversells the NLP; a static example row is honest about what's actually happening (full-sentence parsing on submit, not live suggestion).
- **Errors:** unrecognized/impossible input produces a brief banner with the actual reason (current pattern, correct) — never a silent no-op, never a generic "I didn't understand."

- **Query responses vs. proposal responses vs. errors** are three visually distinct card types (see §9/§10) so a player never mistakes "here's information" for "here's something pending your approval."

---

8. Assisted Mode — Proposal Card

Design intent: the player gave a fairly direct instruction; the card should feel like a confirmation dialog for a real action, not a summary of a conversation.

Card contents, top to bottom:

1. The player's own instruction, shown briefly (e.g., in quotes, small type) — grounds the card in what was actually typed
2. Actor name (who's about to act)
3. The single action, described in plain language ("Attack Goblin 1 with Longbow")
4. A compact checklist of what was validated, each with a check or cross and the real reason if crossed (this is already the current build's strongest pattern — keep it exactly)
5. Approve / Cancel, with Approve visually disabled (not hidden) if any check failed — the player should see *why* they can't approve, not just fail to find the button

Emotional target: "the system understood what I asked and is asking permission" — achieved by (a) echoing the actual input text, and (b) showing real validation rather than a canned "sounds good!" message.

---

9. Adventure Mode — Multi-Step Proposal Card

Same card family as Assisted, extended for sequences:

1. Instruction echoed, same as Assisted
2. **Numbered steps**, each with its own line: description + check/cross + reason if invalid (current build's pattern, correct — keep numbering, don't collapse to a paragraph)
3. **A step's invalidity doesn't halt earlier steps' checkmarks from displaying correctly** — every step is independently validated and independently shown, so the player can see exactly which link in the chain broke, not just "no" for the whole thing

4. **Stale proposals:** if the situation changed between proposal and approval (a target died, moved, etc.), the card transitions to a visually distinct "stale" treatment (current build uses a red-tinted border, which is right) with a **Recalculate** action that re-runs the same instruction against current state and replaces the card — framed as "let me re-check that for you," not as an error the player caused
5. Approve remains disabled whenever any step is invalid; atomic execution (all steps or none) is a backend guarantee the UI reflects by *never* showing a partially-completed state — either the whole card resolves into log entries, or nothing changes and the card stays interactive

---

10. Query / Inspect Experience

The number one design rule here: a query card must be visually impossible to confuse with a proposal card, because a proposal implies "click Approve and something happens" while a query is inert information. Concrete differentiators:

- No Approve/Cancel buttons — a query card has a single "Dismiss," reinforcing that nothing is pending
- A distinct header treatment (e.g., a question-mark glyph or different accent color from the proposal card's action-oriented framing)
- The verdict ("Yes — this is currently valid" / "No — [reason]") stated plainly at the end, after the same itemized-checklist pattern used elsewhere — consistency in *how* things are validated, difference in *what happens after*

Inspect (e.g., "inspect the pillar," "what can I do from here") follows the same card family but is purely descriptive — a list of facts, no verdict line, since there's no yes/no question being answered.

---

11. Session Log

Design goals: compact, readable, and unambiguous about whose action caused what.

- **Turn separators** (current build's "— Round N —" pattern) stay — they're cheap and orient the reader instantly.
- **Player vs. enemy events** should be distinguishable at a glance without reading — a subtle left-edge color tag (matching the party/enemy side-coloring used everywhere

else) rather than a text prefix, keeping the log's actual sentences uncluttered.

- **Critical hits and deaths** deserve slightly stronger typographic weight (not color-flashing, not animation — this is a *log*, restraint matters) so a player scrolling back can find the important beats quickly.
- **Compactness:** one event per line, full sentences (current build's approach), no collapsing of related events into a single line — a hit roll and its damage are two facts, showing both matters more than saving vertical space.
- **Victory/defeat** get a distinct log entry style (see §14 for the accompanying banner) but the log itself should still read as a plain factual record — "The Ruined Crypt encounter is cleared," not celebratory copy inside the log; celebration belongs in the banner treatment, not the historical record.

---

12. Responsive Design

The board is never sacrificed. Everything else negotiates around it.

Desktop/laptop (current three-column layout — keep as baseline): party+actions | board | initiative+log, side by side, all persistently visible. This is correct and should remain the default at generous widths.

Tablet (narrower, but still comfortable touch target size):

- Board stays centered and full-width-available.
- Party panel and action bar **collapse to a persistent bottom bar** (not an overlay) showing the active character's name, HP, and action buttons — always reachable without a tap, because it's the thing the player touches most.
- Initiative strip **moves to a slim horizontal strip above the board** rather than a side column.
- Session log becomes **collapsible** — a pull-tab or small "Log" toggle that expands as a bottom sheet over (not beside) the board when opened, and closes to a single-line "last event" preview when collapsed, so nothing important is ever fully invisible.
- Command bar (Assisted/Adventure) stays **persistent at the very bottom**, above the collapsed party bar, since typing is a primary interaction, not a secondary one.

Small screens (phone-width):

- Board becomes the near-entirety of the screen, pannable/zoomable if the map exceeds comfortable tap-target size at full-fit zoom.

- Party info reduces to a **compact strip**: portrait-sized icon + HP bar only, tap to expand into the full panel as an **overlay** (not inline growth, since inline growth would push the board around).
- Action bar becomes a **bottom sheet that expands on demand** — a single "Actions" tab visible by default (showing just the currently-selected pending action, if any), tapping it reveals the full Move/Attack/Ability/End Turn set as a sheet over the board.
- Turn order reduces to **"You" vs. "Next: [name]"** text, full strip available via tap.
- Log reduces to the **single most recent event**, shown as a thin strip, tap to expand full history as a full-screen overlay.
- Command bar remains reachable via a persistent icon that expands into the input, rather than an always-present text field competing for the limited vertical space with the board.

What must never happen at any width: stacking every desktop panel vertically in original order. That's the anti-pattern the brief explicitly warns against, and it's worth naming directly — a naive "just let everything wrap" approach would push the board itself out of the first viewport on a phone, which fails the core "board is the centerpiece" principle at exactly the width where screen space is scarcest.

---

13. Visual Direction

The current direction (dark walnut wood, parchment tones, brass/gold accents, Cinzel display type + serif body text) is correct and should be extended, not replaced. It already avoids the two named failure modes — it doesn't read as a generic SaaS dashboard, and it isn't yet over-decorated into illegibility. The refinements below are about *consistency and restraint at scale*, not a new aesthetic.

- **Color language:** three functional color families, used consistently everywhere they appear (board, panels, log, cards) — a warm neutral base (wood/parchment), a gold/brass accent reserved for *emphasis and current-turn/selection state only* (not decoration), and two side-colors (party blue-ish, enemy green-ish, as already established) used identically on tokens, panel edges, and log tags so the player learns one color vocabulary instead of several.
- **Panels:** consistent corner treatment, consistent border weight, subtle inset shadow to suggest physical depth (already present) — panels should feel like distinct physical objects on the table (character sheets, a dice tray, a log book), not like app cards.

- **Typography:** display serif (Cinzel or equivalent) reserved for headers/titles only; body serif for everything read at length (log, tooltips, card text) — this split already exists and should be held to strictly as the ability count grows, so tooltips and ability names don't start competing stylistically with section headers.
- **Selection states:** exactly one "this is selected/active" visual language (gold outline/glow) reused everywhere — character panels, action buttons, board tokens — so a player's learned association ("gold means active") transfers between contexts for free.
- **Player/enemy differentiation:** consistent side-coloring as above, reinforced by (not competing with) the selection/targeting color language from §2 and §6.
- **Action feedback:** color and text, not icon-only — a colorblind-safe requirement given how much of this UI's meaning currently lives in color (see §16).

---

14. Animation / Feedback Philosophy

Principle: every animation communicates a state change in the authoritative game state. None are decorative.

Event	Feedback
Selecting a character	Immediate, no-delay visual state change (border/glow) — selection must feel instant, zero animation latency
Turn starts	A brief (under half a second), one-time highlight pulse on the new active character's panel and board token — marks the moment, doesn't linger
Movement	Token animates along the actual path taken (not a teleport-cut), short duration (roughly 150-300ms), so the player's spatial model of the board stays intact
Attack	A brief directional cue (e.g., a short line or flash between attacker and target) synced to the moment the roll result appears — the dice-result panel already does the "reveal the number" job; the board's job is just to make clear *where* the resolution happened
Casting (ability use)	Same directional-cue pattern as attack, differentiated by the ability's associated color (see §6) rather than a different animation *type* — keeps the vocabulary small

Healing	Inverse of the damage cue — a brief upward/restorative visual on the target, paired with the HP bar visibly refilling rather than snapping to the new value
Taking damage	HP bar animates down to the new value (not an instant snap) — the snap is what makes damage feel like a UI update instead of an event; a brief drain animation (a few hundred ms) is enough
Death	Token fades/removes from the board over a short duration rather than vanishing instantly — instant removal reads as a bug, not a defeat
Invalid action	The feedback banner itself is the animation — a quick slide-in/fade, no board-level flash or shake (which would be noisy given how often invalid attempts can reasonably happen while a player learns the ability set)
Proposal approval	The proposal card's own dismissal (fade/collapse) synced with the resulting board animation(s) firing immediately after — approval should feel like it *caused* the thing that happens next, not like two unrelated events
Victory	The one place a slightly more celebratory treatment is earned — a distinct banner (already present conceptually in the current build) with a bit more visual weight than any mid-combat feedback, since it's the rarest, highest-stakes event in the session

Overall restraint rule: if an animation would need to be skippable/disable-able to avoid annoying a player doing many actions per session, it's already too much. Every animation above is sub-second and non-blocking — the player should never wait on an animation to take their next action.

---

15. Accessibility & Readability

- **Never encode meaning in color alone.** Every color-coded state in this document (movement/attack/ability targeting, valid/invalid, player/enemy) needs a paired non-color signal — shape/icon for targeting types, text labels in the log rather than relying on the edge-color tag alone, explicit "Action used" text rather than only a dimmed button.
- **Text contrast** against the wood/parchment palette needs verification at standard WCAG AA thresholds, particularly for the smaller secondary stats (AC, movement) which currently risk being both small and low-contrast.
- **Target sizes** on touch: every board tile and every action button must meet a minimum comfortable tap target (roughly 44px equivalent), which has direct layout consequences for how small the board can shrink before movement/attack targeting

becomes error-prone on a phone — this is a real constraint the responsive design in §12 needs to respect, not just fit content into a smaller box.

- **Screen reader support** is not addressed by this blueprint in detail (out of scope for a visual/interaction spec) but should be flagged to the implementation agent as a follow-up pass — a turn-based, discrete-state game like this is actually a good candidate for reasonable screen-reader support later, since state changes are infrequent and well-defined compared to a real-time game.

---

16. Future Scalability

The interface decisions above are chosen specifically to hold up as the game grows, without requiring UI redesign per addition:

- **More characters:** the party panel is already a repeated component per combatant; scaling from 2 to 4-6 party members means the panel list scrolls or the bottom-bar (tablet/mobile) becomes swipeable — no new panel design needed.
- **More abilities:** directly addressed in §5's two-tier action bar — this is the change most urgently needed relative to the current build, since it's already showing the seam (2 abilities fits fine; 8 would not).
- **More enemy types:** the enemy panel list follows the same repeated-component pattern as party; nothing enemy-type-specific exists in the design.
- **Larger encounters / more complex maps:** the board's pan/zoom behavior (implied for small screens in §12) becomes relevant at desktop size too once maps exceed a single comfortable viewport — worth designing the zoom/pan interaction once, reused everywhere.
- **Conditions / buffs / debuffs:** the character panel has room reserved conceptually (the "current status" line in §3) for a small icon row of active effects, each with the same tooltip-on-hover pattern as abilities — extending the panel, not redesigning it.
- **Inventory / equipment:** would live as a new, secondary panel reachable from the character panel (a tap/click-through), not added to the always-visible primary layout — keeps the core screen uncluttered while the character grows more complex under the hood.
- **Objectives:** a small persistent readout near the encounter name/header (currently just showing the encounter's name) is the natural extension point — "Defeat all enemies" today, other objective text later, same slot.

- **Environmental interactions:** follow the exact targeting pattern already designed in §6 (select an interaction, valid targets highlight, click resolves) — environmental objects are just another target-able thing on the board, not a new interaction paradigm.
- **Campaign play:** out of scope for this single-encounter blueprint, but the encounter-picker concept already present (choosing between Crypt and Training Yard) is the seed of a campaign/encounter-select screen — a grid of encounter cards instead of two buttons, same underlying pattern.

---

17. Recommended Implementation Order

Ordered by (a) how much current-build risk each item resolves, and (b) how much later work depends on it:

1. **Targeting-click-priority fix (§6, §7's occupied-tile note).** Small, isolated, resolves a real interaction gap that undermines trust in the board.
2. **Action-bar two-tier overflow behavior (§5).** Needed before any additional ability is designed as content, or the action bar breaks immediately upon the next addition.
3. **Board targeting color differentiation (§2, §6) — movement vs. attack vs. beneficial-ability vs. harmful-ability as four distinct, consistent hues.** Currently movement and ability-targeting share a color; this is a quick, high-clarity fix.
4. **Active-turn vs. selected-character visual separation (§2, §3).** Prevents the "I deselected the active character and lost track of whose turn it is" confusion.
5. **Query vs. proposal card visual differentiation (§10).** Currently both are the same parchment-card shape; a deliberate header/accent difference is low-effort and closes a real ambiguity.
6. **Responsive tablet layout (§12).** The single biggest scope item in this document; do after the above smaller fixes since it touches every region of the screen.
7. **Responsive phone layout (§12).** Depends on the tablet layout patterns being proven first.
8. **Hover/long-press target preview (§6).** A genuine enhancement, not a fix — schedule after the fixes above, since it's additive rather than corrective.
9. **Accessibility pass (§15).** Should happen once the visual language is stable (after items 1-5), not before, since redesigning colors after an accessibility audit would

mean redoing the audit.

10. **Future-scalability hooks (§16)** — implemented incrementally as each need actually arises (more abilities, conditions, etc.), not built speculatively ahead of the content that would use them.

Summary

The current prototype's instincts are largely right: a wood-and-parchment tabletop, a three-region desktop layout, board-as-centerpiece, itemized real-validation proposal cards. This blueprint's job was to take that proven foundation and specify exactly where it needs sharper edges — a scalable action bar, a cleaner targeting color vocabulary, a genuine responsive strategy, and a firmer visual line between "the system is asking permission" and "the system is telling you something." None of it requires abandoning what already works; all of it is aimed at the same test stated at the top: **does Traditional, Assisted, and Adventure feel like three ways of touching the same table, or three different apps that happen to share a database?**